

Security Model and Security Audit

Lecture 2

Security Model

Part 1

Topics

- ▶ Security Principal Hierarchy
- ▶ Instance Level Security
 - ▶ Logins
 - ▶ Creating a Login
 - ▶ Migrating Logins between Instances
 - ▶ Server Roles
 - ▶ Credentials
- ▶ Database-Level Security
 - ▶ Users
 - ▶ Users with a Login
 - ▶ Users without a Login
 - ▶ Database Roles

Introduction

- ▶ This chapter tackles the security model of SQL Server.
- ▶ SQL Server provides a rich security framework with overlapping layers of security to help counter risks and threats in a manageable way.
- ▶ Important to understand the security model so DBA can implement the technologies in a best fit while minimizing the administration required.
- ▶ Active security refers to practice of limiting the users access to data and structures with permissions.

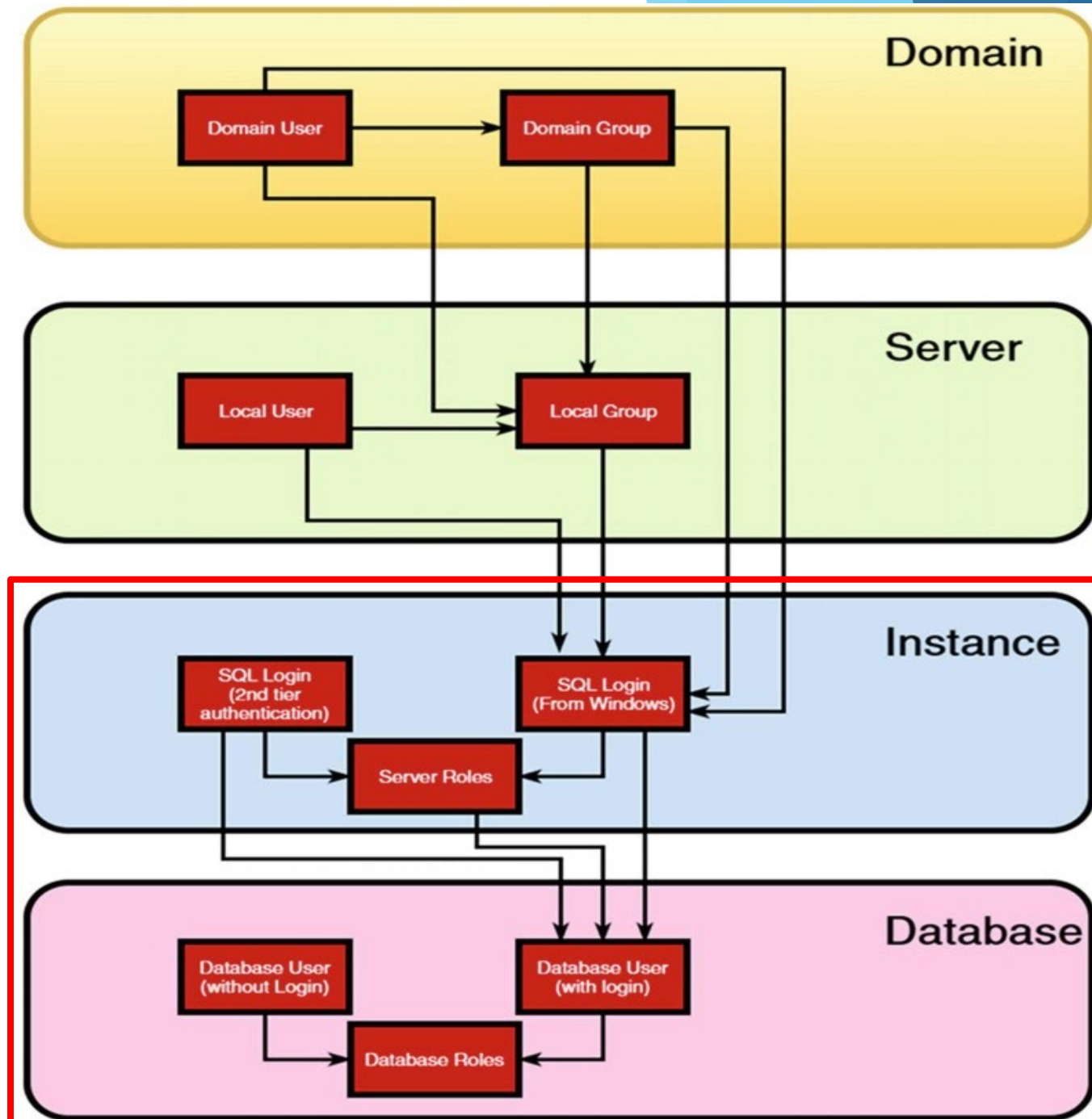
Introduction

Entity	Definition
Principal	A security principal is an entity, such as a user.
Securable	A securable is data, an artifact, or metadata.
Permission	Permissions are rights that are granted or denied to a security principal, to define the principal's permitted access to a securable.

Table 2-1 defines the entities and definitions found in the security model that utilize active security.

Security Principal Hierarchy

- ▶ Security principals are organized in a hierarchy
- ▶ Administrators can assign permissions to groups
- ▶ Figure 2-1 shows complete hierarchy of security principals that can access data or data structures within SQL server.
- ▶ Begins at domain level and ends at the database layer.

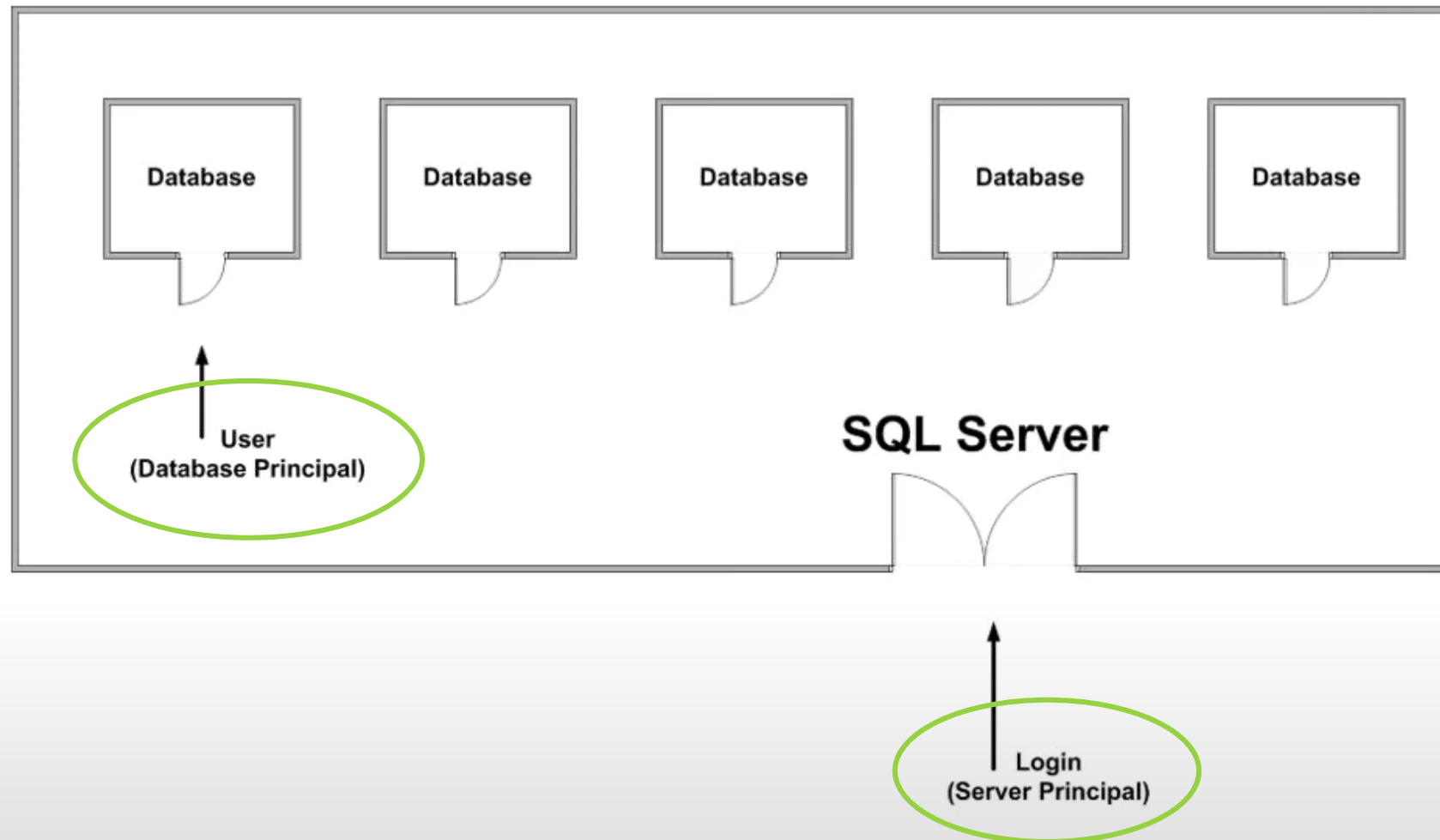




Instance Level Security

- ▶ Implementing instance level security involves creating and managing
 - 1) Logins
 - 2) Credentials → security token
 - 3) Server Roles
- ▶ Securable at the instance level include databases, endpoints and AlwaysOn Availability Groups.
- ▶ Cryptographic Providers and SQL Server Audits are also administered at the instance level.

Instance Level Security





Instance Level Security: Logins

- ▶ Since SQL Server 2012, it's been possible to authenticate directly to a database
- ▶ Database engine users will need to authenticate at the instance level
- ▶ SQL Server supports:

Windows
Authentication

Mixed Mode
Authentication



Instance Level Security: Logins

Windows Authentication

- ▶ Users authenticate to local server or to the domain before accessing the SQL Server Instance.
- ▶ A login is created within the SQL Server Instance that maps to either Windows users or Windows group which contains the Windows user.
- ▶ The Security Identifier (SID) of the Windows principal is stored in the Master database of the instance.
- ▶ In addition, you can also map a login to a certificate or an asymmetric key. Doing so allows for code signing so that permissions to procedures can be abstracted.
- ▶ This helps when using dynamic SQL which breaks the ownership chain. In this scenario, SQL combines the permissions from the user who called the procedure and the user who maps to the certificate.



Instance Level Security: Logins

Mixed Mode Authentication

- ▶ create a 2nd tier login known as SQL Logins.
- ▶ These logins have their name, password and SID stored in the Master database of the instance.
- ▶ Users authenticate directly with this username/password without prior authentication.
- ▶ This is less secure than Windows authentication as it is possible to attack the instance without first authenticating to the domain. Therefore Windows authentication is recommended.
- ▶ However it may be necessary for reasons such as legacy applications, access from outside the domain (Linux) or where security is on the application tier and requires a single SQL Login to connect to the database.

Instance Level Security: Logins

- ▶ Create login using T-SQL via the Create Login statement.
- ▶ Table 2-2 details the WITH options that are valid when creating a login from a Windows security principal.

Option	Description
DEFAULT_DATABASE	Specifies a “landing” database for the Login. This is the database, to which their context will be scoped when they initially authenticate to the instance. This scope can be overwritten in the connection string.
DEFAULT_LANGUAGE	Specifies the default language that will be assigned to the Login. If omitted, then this is configured to be the default language of the instance.

Table 2-2:CREATE LOGIN Options for Windows Security Principal

Instance Level Security: Logins

- ▶ Listing 2-1 demonstrates how to create a Login for Pete in the CarterSecureSafe.com domain.

```
USE master
GO
CREATE LOGIN [cartersecuresafe\Pete]
FROM WINDOWS
WITH DEFAULT_DATABASE=master,
DEFAULT_LANGUAGE=British ;
GO
```

Listing 2-1: Create a Login from a Windows Security Principal

Instance Level Security: Logins

When creating a 2nd tier, SQL Login, there are many more WITH options that can be used.

These options are described in Table 2-1.

Option	Description
PASSWORD	Specifies the initial password that will be used by the Login, in clear text
PASSWORD HASHED	Specifies a hashed representation of the initial password that will be used by the Login*
SID	Specifies the SID of the Login*
DEFAULT_DATABASE	Specifies a “landing” database for the Login. This is the database, to which their context will be scoped when they initially authenticate to the instance. This scope can be overwritten in the connection string.
DEFAULT_LANGUAGE	Specifies the default language that will be assigned to the Login. If omitted, then this is configured to be the default language of the instance.
CHECK_POLICY	Specifies that the Login’s password must meet the same requirements, such as length and complexity, as Windows users, as enforced by Group Policy, or Local Security Policy.
CHECK_EXPIRATION	Specifies that the Login’s password will expire, in line with password expiration policy configured for Windows users, as enforced by Group Policy or Local Security Policy. This option is only valid if CHECK_POLICY is also specified.
MUST_CHANGE	Specifies that the user must change their password the first time that they log in to the instance. This option can only be used if CHECK_EXPIRATION is also specified.

Table 2-3 CREATE LOGIN Options for SQL Login

Instance Level Security: Logins

- ▶ Listing 2-2 demonstrates how to create a SQL Login called Danni.

```
USE master
GO
CREATE LOGIN Danni
WITH PASSWORD='C0mplexPa$$w0rd',
DEFAULT_DATABASE=master, CHECK_EXPIRATION=OFF,
CHECK_POLICY=ON ;
GO
```

Listing 2-2 Create a SQL Login

The password must meet the complexity requirements specified by Group Policy, but the password will not expire.

Instance Level Security: Logins

- ▶ Alternatively, logins can be created in SQL Server Management Studio.
- ▶ Figure 2-2 shows how to create a 2nd tier Login for Reuben in the Object Explorer.
- ▶ In the next section of the General page, the Windows policies can be specified to apply to the Login.
- ▶ These policies are cumulative.
- ▶ At the bottom of the screen, a default database for the login to land in and the default language should be configured.

The screenshot displays the 'Login - New' dialog box with the 'General' page selected. The 'Login name' is 'Reuben'. 'SQL Server authentication' is selected. Passwords are masked with dots. 'Enforce password policy' and 'Enforce password expiration' are checked. 'User must change password at next login' is unchecked. Authentication methods include 'Mapped to certificate', 'Mapped to asymmetric key', and 'Map to Credential'. The 'Default database' is 'master' and the 'Default language' is '<default>'. The 'Progress' section shows 'Ready'.

Mapped Credentials	
Credential	Provider

Figure 2-2 New Login Dialog Box–General Page



Instance Level Security: Migrating Logins between Instances

- ▶ In some cases, the same Login must be used on multiple servers.
- ▶ For Logins from Windows security principals, no problem.
- ▶ With SQL Logins, this scenario is more challenging. During the move, database users will become orphaned with no map to a login, because of different SID.
- ▶ Use ALTER USER statement WITH LOGIN option. Listing 2-3 demonstrates this for a user Danni.

```
USE WideWorldImporters  
GO  
ALTER USER Danni WITH LOGIN = Danni ;
```

Listing 2-3 Remap a Database User to a Login

Instance Level Security: Migrating Logins between Instances

- ▶ Another, more efficient way is to create a Login and manually assign it with the correct SID during migration.
- ▶ The WITH SID option can be used in a script. Such a script can be used in SQLCMD mode, which is shown in Listing 2-4 that replicates the login for Danni from ProdInstance1 to DRInstance1 on CarterSecureSafe's server.
- ▶ Note that this script must be run in SQLCMD mode, not in management studio.

```
:CONNECT CARTERSECURESAFE\ProdInstance1
DECLARE @SQL NVARCHAR(MAX) ;
SET @SQL = (SELECT 'CREATE LOGIN '
              + name
              + ' WITH PASSWORD =
"C0mplexPassw0rd", SID = 0x'
              +
CONVERT(NVARCHAR(64), SID, 2)
              FROM sys.sql_logins
              WHERE Name = 'Danni') ;
:CONNECT CARTERSECURESAFE\DRInstance1
EXEC (@SQL) ;
```

Listing 2-4Migrate a Login to a New Instance



Instance Level Security: Migrating Logins between Instances

► Caution!!

- Password may be sent in cleartext depending on network configuration.
- Hash the password for additional security.
- Use the password_hash column of sys.sql_logins view or using the Hashbytes() function.

Instance Level Security: Migrating Logins between Instances

Listing 2-5 shows how this is done using DDL and can be placed in script control or other environments when required.

```
DECLARE @password NVARCHAR(MAX) = 'C0mplexPa$$w0rd' ;
DECLARE @salt VARBINARY(4) = CRYPT_GEN_RANDOM(4) ;
DECLARE @hash VARBINARY(1000) ;
DECLARE @SQL NVARCHAR(MAX) ;
SET @hash = (SELECT 0x0200 + @salt + HASHBYTES('SHA2_512', CAST(@password AS
VARBINARY(MAX)) + @salt)) ;
SET @SQL = (SELECT 'CREATE LOGIN '
              + Name
              + ' WITH PASSWORD = '
              + CONVERT(NVARCHAR(1000), @hash, 1)
              + ' HASHED, SID = 0x'
              + CONVERT(NVARCHAR(64), SID, 2)
            FROM sys.sql_logins
            WHERE is_disabled = 0
            FOR XML PATH('')) ;

SELECT @SQL ;
```

Listing 2-5 Script Out Logins with Hashed Passwords



Instance Level Security: Migrating Logins between Instances

- ▶ Hashbytes() returns a hash of its input
- ▶ It uses CRYPT_GEN_RANDOM() which is part of Windows CAPI that generates a random hex number.
- ▶ Table 2-4 and 2-5 shows the details.

Parameter	Description
Algorithm	The algorithm that the function will use to hash the data. Acceptable values are: • MD2 • MD4 • MD5 • SHA • SHA1 • SHA2_256 • SHA2_512
Input	The value that will be hashed by the function

Table 2-4 HASHBYTES() Parameters

Parameter	Description
Length	The length of the number to be generated
Seed	An optional randomization seed

Table 2-5 CRYPT_GEN_RANDOM Parameters



Instance Level Security: Server Roles

- SQL Server provides built in server roles
- These allow assignment of instance level permissions to Logins with common requirements.
- It is not possible to change permissions for these roles.
- The roles are stated in Table 2-6.

Role	Description
Sysadmin	The <code>sysadmin</code> role gives administrative permissions to the entire instance. A member of the <code>sysadmin</code> role can perform any action within the instance of the SQL Server relational engine.
Blkadmin	In conjunction with the <code>INSERT</code> permission on the target table within a database, the <code>bulkadmin</code> role allows a user to import data from a file using the <code>BULK INSERT</code> statement. This role is normally given to service accounts that run ETL processes.
Dbcreator	The <code>dbcreator</code> role allows its members to create new databases within the instance. Once a Login creates a database, that Login is automatically the owner of that database and is able to perform any action inside it.
Diskadmin	The <code>diskadmin</code> role gives its members the permissions to manage backup devices within SQL Server.
Processadmin	Members of the <code>processadmin</code> role are able to stop the instance from T-SQL or SSMS (SQL Server Management Studio). They are also able to kill running processes.
Public	All Logins are added to the <code>public</code> role. Although you can assign permissions to the public role, this does not fit with the principle of least privilege. This role is normally only used for internal SQL Server operations, such as authentication to <code>TempDB</code> .
Securityadmin	Members of the <code>securityadmin</code> role are able to manage Logins at the instance level. For example, members may add a Login to a server role (except <code>sysadmin</code>) or assign permissions to an instance-level resource, such as an endpoint. However, they cannot assign permissions within a database-to-database users.
Serveradmin	<code>serveradmin</code> combines the <code>diskadmin</code> and <code>processadmin</code> roles. In addition to being able to start or stop the instance, however, members of this role can also shut down the instance using the <code>SHUTDOWN</code> T-SQL statement. The subtle difference here is that the <code>SHUTDOWN</code> command gives you the option of not running a <code>CHECKPOINT</code> in each database if you use it with the <code>NOWAIT</code> option. Additionally, members of this role can alter endpoints and view all instance metadata.
setupadmin	Members of the <code>setupadmin</code> role are able to create and manage linked servers.

Table 2-6 Fixed Server Roles



Instance Level Security: Server Roles

- Custom server roles are also possible.
- This allows customizable permissions to Logins
- Listing 2-6 demonstrates how to do this and grant the relevant permissions
- There are 3 commands related to permissions
 1. GRANT - provides permission
 2. DENY -denies permission
 3. REVOKE - removes a permission



Instance Level Security: Server Roles

```
CREATE SERVER ROLE AVAILABILITYROLE AUTHORIZATION  
[CarterSecureSafe\SQLAdmin] ;  
GO  
GRANT ALTER ANY AVAILABILITYROLE GROUP TO AVAILABILITYROLE ;  
GRANT ALTER ANY ENDPOINT TO AVAILABILITYROLE ;  
GRANT CREATE AVAILABILITYROLE GROUP TO AVAILABILITYROLE ;  
GRANT CREATE ENDPOINT TO AVAILABILITYROLE ;  
GO
```

Listing 2-6 Create Server Role and Grant Permissions

```
ALTER SERVER ROLE AvailabilityRole ADD MEMBER Danni ;  
GO
```

Listing 2-7 Add Login to Server Role

Instance Level Security: Server Roles

- Lastly, server roles can also be managed through the Login Properties dialog box in SSMS.
- Figure 2-3 shows the login properties box for Reuben.

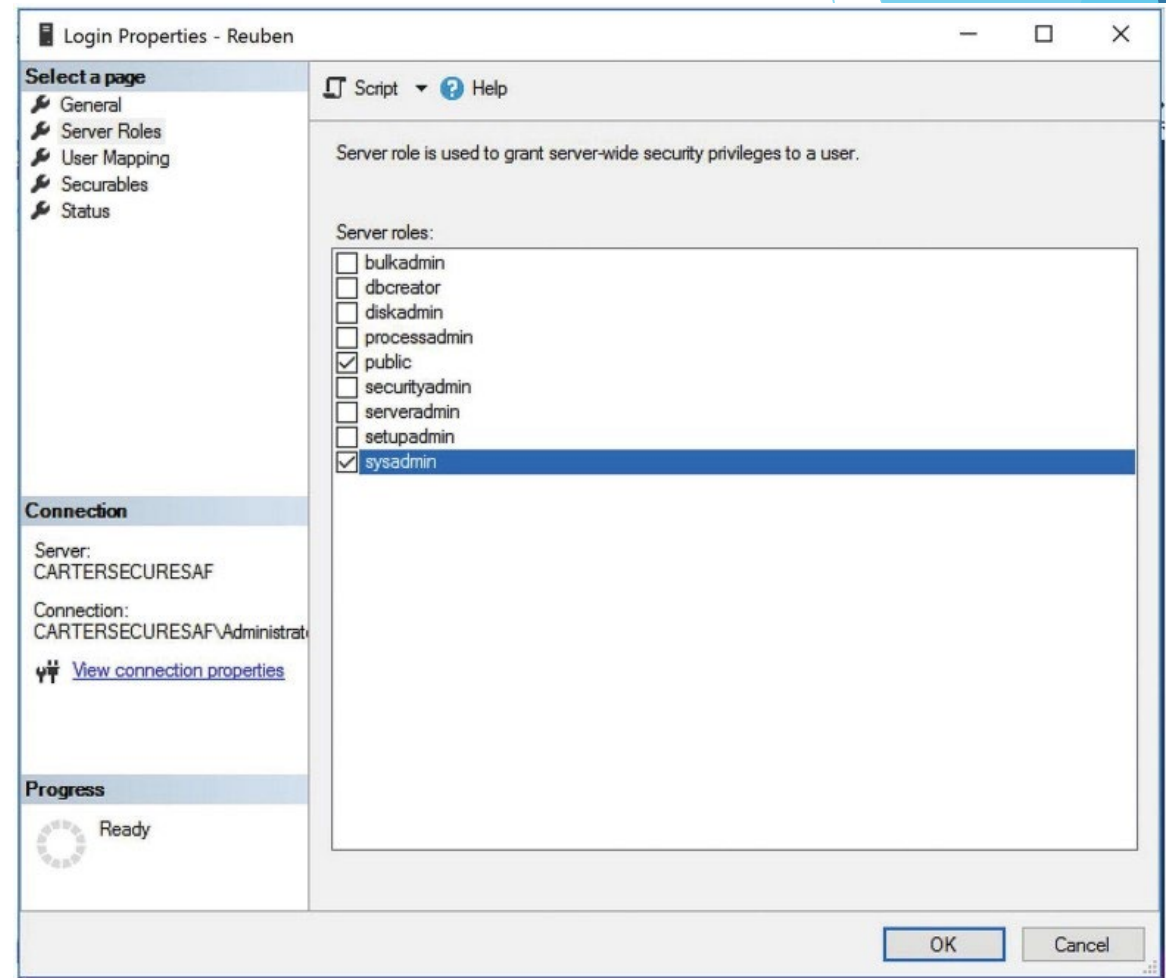


Figure 2-3 Login properties-server roles



Instance Level Security: Credentials

- Provide ability to access resources that are external to the SQL Server instance, including system level resources.
- When used to access OS level resources, a credential will record the identity and password of a Windows security principal.
- If used for backups to Azure, the Credential will record the name and private key of the Azure storage account.
- Listing 2-8 demonstrates creating a Credential for backups to a storage account on Azure called CarterSecureSafeStorageAcc

```
CREATE CREDENTIAL URLBackupCredential
WITH IDENTITY = 'CarterSecureSafeStorageAcc'
, SECRET
= '\Ydfg\SGdTgJNpVF1992sBv7Bp1gyL61I33wNrTMHGBDdtVcx97F5f6SC5uD159FeY2/Ijx
yqsuLU2xrkrNAGT==' ;
```

Listing 2-8 Create a Credential for Backups to Azure



Data Level Security - Users

- ▶ At the database level, security is implemented by assigning permissions to security principals - users and database roles.
- ▶ A database user will typically be created from a Login at the instance level. This means that the same instance level security principal can be granted permissions on resources in multiple databases.
- ▶ Since SQL Server 2012, however, it has also been possible to create a user without a Login.



Data Level Security - Users With a Login

- ▶ Users can be created by using the CREATE USER T-SQL statement.
- ▶ Users that are created with an association to a Login have a limited set of options that can be configured.
- ▶ These options are detailed in Table 2-7.

Option	Description
DEFAULT_SCHEMA	Specifies the default schema for a user.
ALLOW_ENCRYPTED_VALUE_MODIFICATIONS	Specifies that users will be allowed to bulk copy encrypted data, without first decrypting it.

Table 2-7 Options When Creating a User From a Login



Data Level Security - Users With a Login

- ▶ Listing 2-9 shows how to create a user in the WideWorldImporters database for the Login called Danni
- ▶ The script creates a user that has the same name as the Login. This helps security principal administration and makes the hierarchy of principals transparent.
- ▶ The script sets the user's default schema to be Sales so Danni will be able to reference objects in the Sales schema using one-part names.
- ▶ If no default schema is specified, then default schema will be dbo.

```
USE WideWorldImporters
GO
CREATE USER Danni FOR LOGIN Danni
    WITH DEFAULT_SCHEMA = Sales ;
```

Listing 2-9 Create a User From a Login

Data Level Security - Users With a Login

Method 1: Use the GUI to create a database user. Go through Databases->WideWorldImporters->Security in Object Explorer, then open the New User dialog box by selecting New from user's menu.

Method 2: Use the Login Properties dialog box :Go to Security->Logins in Object Explorer, double-click the Reuben Login and select User Mapping page. This page is illustrated in Figure 2-4.

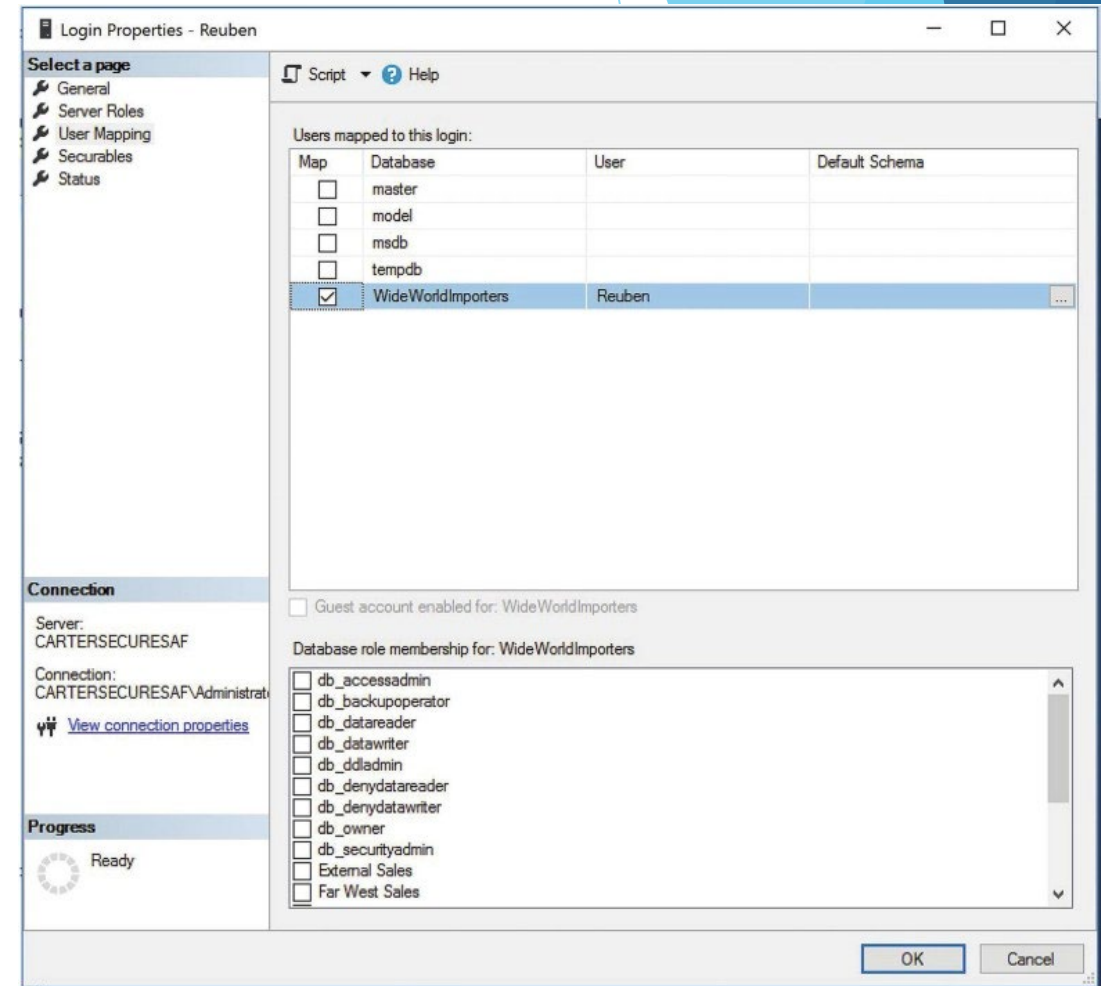


Figure 2-4 Login properties–user mapping

Data Level Security - Users Without a Login

- ▶ When creating a user that is not associated with a Login, the user can either be mapped to a Windows security principal, or it can be created using SQL Server authentication.
- ▶ Users can only be created using SQL Server authentication if the database is configured with a containment level of Partial.
- ▶ Table 2-8 details the properties that can be configured when creating a user without a Login.
- ▶ Listing 2-10 demonstrates how to create a user called Phil from a Windows security principal in the Cartersecursafe domain.

Option	Description
DEFAULT_SCHEMA	Specifies the default schema for a user.
ALLOW_ENCRYPTED_VALUE_MODIFICATIONS	Specifies that users will be allowed to bulk copy encrypted data, without first decrypting it.
DEFAULT_LANGUAGE	Specify the default language for the user. This option can be expressed as an lcid, a language name, or a language alias.
SID	Specify the SID that will be associated with the user (SQL Server authentication only). This can be used to create users in multiple databases, that share the same SID. This can help with high availability and disaster recovery techniques, using AlwaysOn Availability Groups.

Table 2-8 Options When Creating a User Without a Login

```
USE WideWorldImporters
GO
CREATE USER [cartersecursafe\phil]
    WITH DEFAULT_SCHEMA=dbo ;
```

Listing 2-10 Create a User From a Windows Security Principal

Data Level Security - Users Without a Login

- ▶ Creating users with SQL Authentication is only possible with contained database.
- ▶ Otherwise, the error displayed in Figure 2-5 will be received.

A contained database is a database that is isolated from other databases and from the instance of SQL Server/ SQL Database (and the master database) that hosts the database.

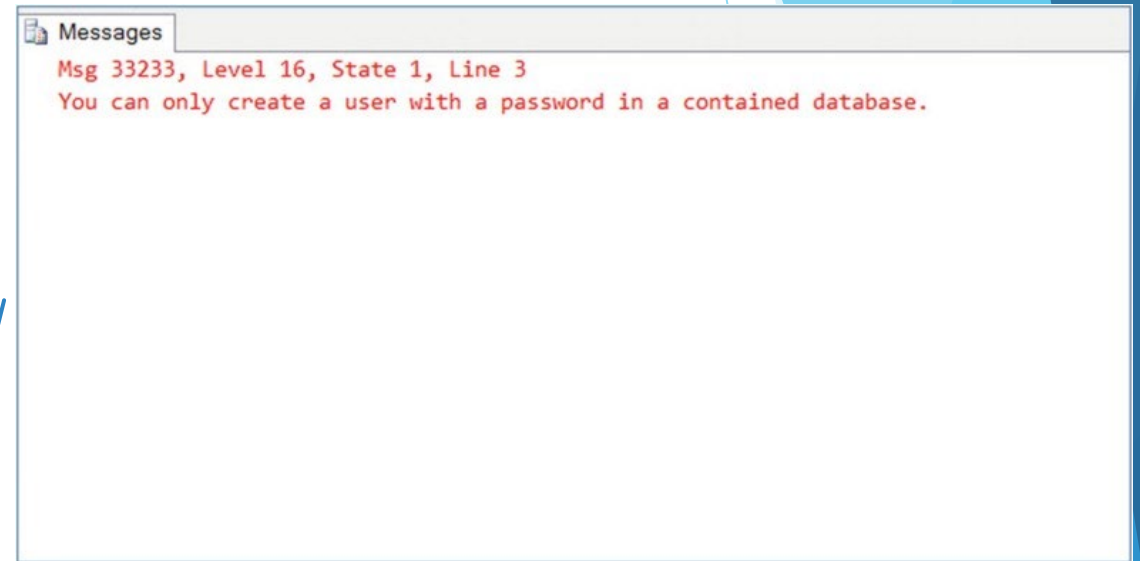


Figure 2-5 Error when attempting to create a user with password in a non-contained database

Data Level Security - Users Without a Login

- ▶ The script in Listing 2-11 configures the instance to allow contained database authentication
- ▶ Then it configures the database to support partial containment.
- ▶ Finally, the script creates a user Pete with a password.

```
USE master
GO
EXEC sp_configure 'show advanced options', 1 ;
GO
RECONFIGURE ;
GO
EXEC sp_configure 'contained database
authentication', '1' ;
GO
RECONFIGURE WITH OVERRIDE ;
ALTER DATABASE WideWorldImporters
    SET CONTAINMENT = PARTIAL
    WITH NO_WAIT ;
GO
USE WideWorldImporters
GO
CREATE USER Pete
    WITH PASSWORD = 'Pa$$w0rd123' ;
GO
```

Listing 2-11 Create a User With Password in a Contained Database



Data Level Security - Users Without a Login

▶ Extra considerations:

- ▶ Some applications may require a user have permissions to multiple databases.
- ▶ If the user is mapped to a Windows user or group, then straightforward: SID=the Windows object.
- ▶ If the database user is using 2nd-tier authentication - duplicate the SID of the user from the first database. To do this:
 1. Create a user with password in the first database.
 2. Retrieve the user's SID from `sys.database_principals`
 3. Create the user in additional databases, specifically supplying the SID that you have recovered from the metadata.
- ▶ The `sys.database_principals` catalog view exposes the columns detailed in Table 2-9.



Data Level Security - Users Without a Login

Column	Description
name	The name of the security principal
principal_id	The ID of the security principal. This ID is only unique within the database.
type	A single character abbreviation of the type description
type_desc	A description of the type of security principal. Possible values are: • APPLICATION_ROLE • CERTIFICATE_MAPPED_USER • EXTERNAL_USER • WINDOWS_GROUP • ASYMMETRIC_KEY_MAPPED_USER • DATABASE_ROLE • SQL_USER • WINDOWS_USER • EXTERNAL_GROUPS
default_schema_name	The name of the principals default schema
create_date	The date and time that the principal was created
modify_date	The date and time that the principal was last modified
owning_principal_id	The ID of the security principal that is marked as the owner of the principal. This will be 1, which is the ID of dbo, for all principals except database roles.
sid	The SID of the security principal
is_fixed_role	Indicates if the principal is a fixed database role. • 1 indicates that the principal is a fixed database role. • 0 indicates that the principal is not a fixed database role.
authentication_type	Indicates how the principal authenticates to the database. Possible values are listed below: • 0 indicates No authentication. • 1 indicates Instance authentication. • 2 indicates Database authentication. • 3 indicates Windows Authentication.
authentication_type_desc	A textual description of the authentication type. Possible values are: • NONE • INSTANCE • DATABASE • WINDOWS
default_language_name	The name of the default language assigned to the principal
default_language_lcid	The lcid of the default language assigned to the principal

Table 2-9
sys.database_principals Columns



Database Roles

- ▶ SQL Server provides a set of built-in database roles, out-of-the-box.
- ▶ Fixed database roles: allow database-level permissions assignment to users with requirements.
- ▶ Not possible to change the permissions that are granted;
- ▶ Can only add and remove users to and from the roles.
- ▶ Table 2-10 details each of these roles.

Database Roles

Role	Description
db_accessadmin	Members of this role can add and remove database users from the database.
db_backupoperator	The <code>db_backupoperator</code> role gives users the permissions they need to back up the database, natively. It may not work for third-party backup tools, such as CommVault or Backup Exec, since these tools often demand <code>sysadmin</code> rights.
db_datareader	Members of the <code>db_datareader</code> role can run <code>SELECT</code> statements against any table in the database. It is possible to override this for specific tables by explicitly denying a user, the permissions to read those tables. <code>DENY</code> always overrides <code>GRANT</code>
db_datawriter	Members of the <code>db_datawriter</code> role can perform DML (Data Manipulation Language) statements against any table in the database. It is possible to override this for specific tables by specifically denying a user the permissions to write to a table. <code>DENY</code> will always override <code>GRANT</code> .
db_denydatareader	The <code>db_denydatareader</code> role denies the <code>SELECT</code> permission against every table in the database.
db_denydatawriter	The <code>db_denydatawriter</code> role denies its members the permissions to perform DLM statements against every table in the database.
db_ddladmin	Members of this role are given the ability to run <code>CREATE</code> , <code>ALTER</code> , and <code>DROP</code> statements against any object in the database. This role is rarely used, but I have seen a couple of examples of poorly written applications that create database objects on the fly. If you are responsible for administering an application such as this, then the <code>ddl_admin</code> role may be useful.
db_owner	Members of the <code>db_owner</code> role can perform any action within the database that has not been specifically denied.
db_securityadmin	Members of this role can <code>GRANT</code> , <code>DENY</code> , and <code>REVOKE</code> a user's permissions to securables. They can also modify role memberships, with the exception of the <code>db_owner</code> role.

Table 2-10
Fixed Database
Roles



Database Roles

- ▶ user-defined database roles: allows creation of roles that map to requirements of business teams using a specific database.

```
USE WideWorldImporters
GO
--Create the role
CREATE ROLE SalesRole AUTHORIZATION dbo ;
GO
--Grant permissions to the role
GRANT DELETE ON SCHEMA::Sales TO SalesRole ;
GRANT INSERT ON SCHEMA::Sales TO SalesRole ;
GRANT SELECT ON SCHEMA::Sales TO SalesRole ;
GRANT UPDATE ON SCHEMA::Sales TO SalesRole ;
--Add user to the role
ALTER ROLE SalesRole ADD MEMBER Danni ;
```

Listing 2-13 Create SalesRole



Summary

- ▶ SQL Server provides a flexible hierarchy to implement security.
 - ▶ Role-based security is available out-of-the-box,
 - ▶ database administrators should embrace this to simplify the administration of security.
- ▶ The database engine supports two methods of authentication:
 - ▶ Windows authentication - SQL Server's implementation of 2nd-tier authentication.
 - ▶ SQL Server authentication - for legacy connections
 - ▶ Windows authentication should be used, unless there is a good reason not to, as it is more secure.
- ▶ Database engine users will typically authenticate to the instance via a Login. This Login will then map to users within the databases to which they require access.
 - ▶ It is also possible, however, for users without Login to authenticate directly to a database via Windows authentication
 - ▶ if the database is contained, then SQL Server authentication can alternatively be used.

Security Audit

Part 2

Topics

- ▶ Understanding SQL Server Audit
 - ▶ SQL Server Audit Actions and Action Groups
- ▶ Implementing SQL Server Audit
 - ▶ Creating a Server Audit
 - ▶ Create a Server Audit Specification
 - ▶ Create a Database Audit Specification
- ▶ Creating Custom Audit Events
 - ▶ Creating the Server Audit and Database Audit Specification
 - ▶ Raising the Event



Introduction

- ▶ Passive Security - practice of logging user activity in order to avoid the threat of non-repudiation
- ▶ SQL Server provides SQL Server Audit to assist with implementing passive security.



Understanding SQL Server Audit

- ▶ Offers DBAs ability to capture granular information at instance and database level.
- ▶ **Save audit Logs to File, Security log or Application log** - one target with each audit. For Application log, any Windows user can access it. Security log is more secure but more complex to configure.
- ▶ **Server Audit resides at instance level** - defines properties of audit and target. Can create multiple audits each instance.
- ▶ Size of log is also an issue as it will increase with time. Policies should be determined on how the log will be cycled and archiving of logs.
- ▶ One Audit can be associated with one or more Server Audit Specifications or Database Audit Specifications. Specifications define the activity captured by the audit at respective levels.



Understanding SQL Server Audit

- ▶ SQL Server Audit Actions and Action Groups
- ▶ Events are based on **SQL Server Event Classes**, with related actions grouped together into Audit Action Groups.
- ▶ These Action Groups map to SQL Server Event Class Categories.
- ▶ When creating specifications, configure to capture Audit Action Groups which contain the events to capture.
- ▶ Audit Groups are available at Server/Instance, Database and Audit layers. Details here <https://docs.microsoft.com/en-us/sql/relational-databases/security/auditing/sql-server-audit-action-groups-and-actions?view=sql-server-ver15>



Implementing SQL Server Audit

Create A Server Audit

- ▶ Can either use GUI or T-SQL's Data Definition Language (DDL) statement with options from Table 3-4.
- ▶ Possible to use filters also on server audits, partitioning it into subset of information.



Implementing SQL Server Audit

Table 3-4 Server Audit Options

Option	Description
FILEPATH	Specifies the file path, where the audit logs will be generated. Only applies if you choose a file target.
MAXSIZE	Specifies the largest size that the audit file can grow to. The minimum size you can specify for this is 2MB. Only applies if you choose a file target.
MAX_ROLLOVER_FILES	When the audit file becomes full, you can either cycle that file or generate a new file. The MAX_ROLLOVER_FILES setting controls how many new files can be generated before they begin to cycle. The default value is UNLIMITED, but specifying a number caps the number of files to this limit. If you set it to 0, then there will only ever be one file, and it will cycle every time it becomes full. Any value above 0 indicates the number of rollover files that will be permitted. So, for example, if you specify 5, then there will be a maximum of six files in total. Only applies if you choose a file target.
MAX_FILES	As an alternative to MAX_ROLLOVER_FILES, the MAX_FILES setting specifies a limit for the number of audit files that can be generated, but when this number is reached, the logs will not cycle. Instead, the audit fails and events that cause an audit action to occur are handled based on the setting for ON_FAILURE. Only applies if you choose a file target.
RESERVE_DISK_SPACE	Pre-allocate space on the volume equal to the value set in MAXSIZE, as opposed to allowing the audit log to grow as required. Only applies if you choose a file target.
QUEUE_DELAY	Specifies if audit events are written synchronously or asynchronously. If set to 0, events are written to the log synchronously. Otherwise, specify the duration in milliseconds that can elapse before events are forced to write. The default value is 1,000 (1 second), which is also the minimum value.
ON_FAILURE	Specifies what should happen if events that cause an audit action fail to be audited to the log. Acceptable values are CONTINUE, SHUTDOWN, or FAIL_OPERATION. When CONTINUE is specified, the operation is allowed to continue. This can lead to unaudited activity occurring. FAIL_OPERATION causes auditable events to fail, but allows other actions to continue. SHUTDOWN forces the instance to stop if auditable events cannot be written to the log.
AUDIT_GUID	Because server and database audit specifications link to the server audit through a GUID, there are occasions when an audit specification can become orphaned. These include when you attach a database to an instance, or when you implement technologies such as AlwaysOn Availability Groups. This option allows you to specify a specific GUID for the server audit, as opposed to having SQL Server generate a new one.

Implementing SQL Server Audit

Create A Server Audit

- ▶ Listing 3-1 demonstrates creating server audit using T-SQL.
- ▶ Enabling the audit is shown in Listing 3-2.

```
USE Master
GO
CREATE SERVER AUDIT [Audit-CarterSecureSafe]
TO FILE
(
    FILEPATH = 'C:\AuditFiles\Audit\'
    ,MAXSIZE = 256 MB
    ,MAX_ROLLOVER_FILES = 2147483647
    ,RESERVE_DISK_SPACE = OFF
)
WITH
(
    QUEUE_DELAY = 1000
    ,ON_FAILURE = CONTINUE
)
WHERE object_name = 'sysadmin' ;
```

Listing 3-1 Create a Server Audit

```
ALTER SERVER AUDIT [Audit-CarterSecureSafe]
WITH (STATE = ON) ;
```

Listing 3-2 Enabling an Audit

Implementing SQL Server Audit

Create A Server Audit

- ▶ Figure 3-1 shows the alternative method of creating using GUI, through the Security Object and under New Audit in Audits object.

Create Audit

Ready

Select a page

- General
- Filter

Connection

CARTERSECURESAF
[CARTERSECURESAF\Administrator]

[View connection properties](#)

Progress

Ready

Script | **Help**

Audit name: Audit-CarterSecureSafe

Queue delay (in milliseconds): 1000

On Audit Log Failure:

- ☒ Continue
- ☐ Shut down server
- ☐ Fail operation

Audit destination: File

File path: c:\auditfiles\audit

Audit File Maximum Limit:

- ☒ Maximum rollover files:
 - ☒ Unlimited
- ☐ Maximum files:
 - Number of files: 2147483647

Maximum file size: 256 ☒ MB ☐ GB ☐ TB

☐ Unlimited

☐ Reserve disk space

OK Cancel Help

Figure 3-1 Create Audit dialog box-general page

Implementing SQL Server Audit

Create A Server Audit Specification

- ▶ Specifications shows what to audit for.
- ▶ As usual, can create using T-SQL or using GUI.
- ▶ Table 3-5 shows the Server Audit Specification Options.
- ▶ Listing 3-3 shows the T-SQL method of creating a Server Audit Specification to capture changes to membership of Server Roles
- ▶ Figure 3-3 shows how to do this using GUI in SSMS.

Argument	Description
audit_specification_name	The name to be assigned to the Server Audit Specification.
audit_name	The name of the Server Audit, to which the specification will be associated.
audit_action_group_name	The name of a group of related auditable actions at the instance level.
STATE	Specifies if the Server Audit Specification should be started on creation.

Table 3-5 Server Audit Specification Options

Implementing SQL Server Audit

```
CREATE SERVER AUDIT SPECIFICATION  
[ServerAuditSpecification-CarterSecureSafe]  
FOR SERVER AUDIT [Audit-CarterSecureSafe]  
ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP) ;
```

Listing 3-3 Create a Server Audit Specification

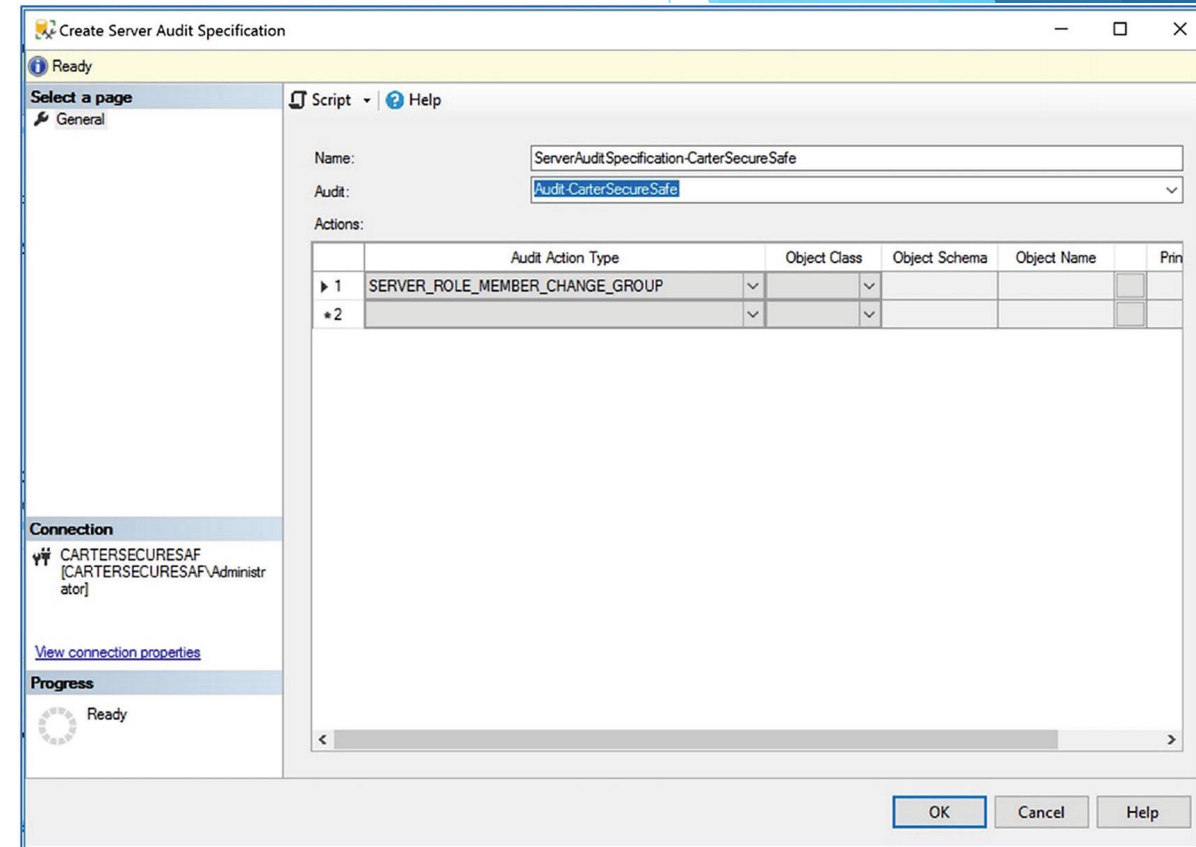


Figure 3-3 Create Server Audit Specification dialog box



Implementing SQL Server Audit

Create A Database Audit Specification

- ▶ Similar to server audit specifications but at database-level, with more flexibility to specify filters such as for securables or principles to be audited.
- ▶ Similarly can use T-SQL or GUI to create this specification.
- ▶ Table 3-6 shows the options available.
- ▶ Listing 3-4 shows the T-SQL demonstration while Figure 3-4 shows how to do it using GUI.

Argument	Description
audit_specification_name	The name to be assigned to the Database Audit Specification
audit_name	The name of the Server Audit, to which the specification will be associated
action	The granular action to be audited
audit_action_group_name	The name of a group of related auditable actions at the database level
class	The class name of the securable
securable	The name of the securable to be audited
principal	The name of the principal to be audited
STATE	Specifies if the Server Audit Specification should be started on creation.

Table 3-6 Database Audit Specification Options

Implementing SQL Server Audit

```
CREATE DATABASE AUDIT SPECIFICATION  
[DatabaseAuditSpecification-  
WideWorldImporters]  
FOR SERVER AUDIT [Audit-  
CarterSecureSafe]  
ADD (DELETE ON OBJECT::Sales.Orders  
BY public) ;
```

Listing 3-4 Create a Database Audit Specification

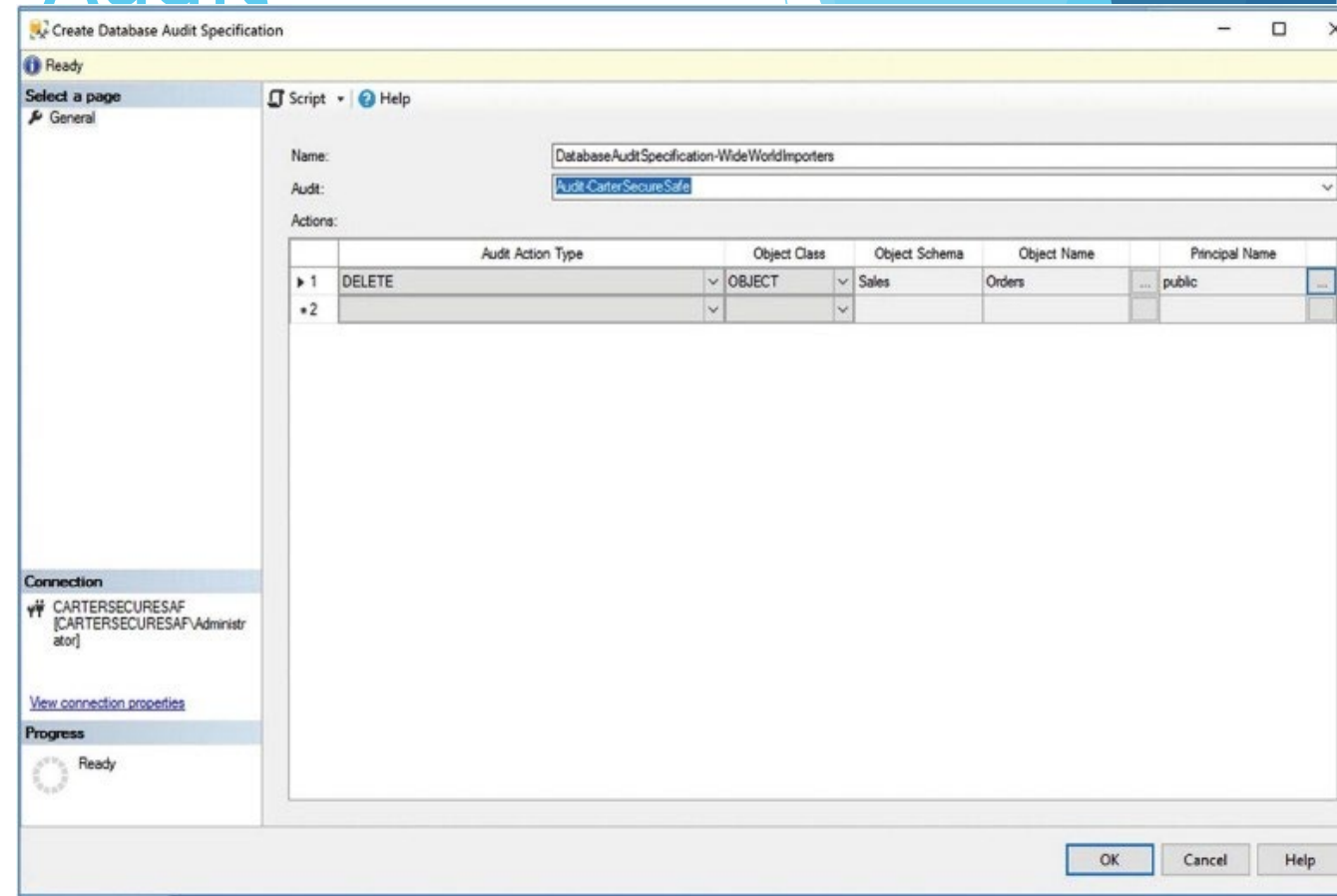


Figure 3-4 Create Database Audit Specification dialog box



Enabling Audit Specifications

- ▶ Audit specifications are off by default, but can be turned on during creation or by altering the specification. Listing 3-5 shows how this is done.

```
ALTER SERVER AUDIT SPECIFICATION  
[DatabaseAuditSpecification-  
WideWorldImporters]  
WITH (STATE = ON) ;
```

Listing 3-5 Enabling An Audit Specification



Creating Custom Audit Events

- ▶ Sometimes may need to capture specific events not possible using out-of-the-box functionality of SQL Server Audit.
- ▶ Use an Audit Specification with the USER_DEFINED_AUDIT_GROUP Audit Action Group and then manually fire the event in application code.
- ▶ The following section demonstrates how to do this to log sales orders with more than five different items ordered.
- ▶ First create the Server Audit and a Database Audit Specification in the WideWorldImporters database where Sales.Orders table is hosted as in Listing 3-6.

Creating Custom Audit Events

```
USE Master
GO
CREATE SERVER AUDIT [Audit-CarterSecureSafeCustom]
TO FILE
(
    FILEPATH = 'c:\audit_files\audit-custom'
    ,MAXSIZE = 256 MB
    ,MAX_ROLLOVER_FILES = 2147483647
    ,RESERVE_DISK_SPACE = OFF
)
WITH
(
    QUEUE_DELAY = 1000
    ,ON_FAILURE = CONTINUE
) ;
GO
CREATE SERVER AUDIT SPECIFICATION [ServerAuditSpecification-CarterSecureSafeCustom]
FOR SERVER AUDIT [Audit-CarterSecureSafeCustom]
ADD (USER_DEFINED_AUDIT_GROUP) ;
GO
ALTER SERVER AUDIT [Audit-CarterSecureSafeCustom]
WITH (STATE = ON) ;
ALTER DATABASE AUDIT SPECIFICATION [ServerAuditSpecification-CarterSecureSafeCustom]
WITH (STATE = ON) ;
```

Listing 3-6 Create the Server Audit and Database Audit Specification



Creating Custom Audit Events

- ▶ Custom events can be raised using the `sp_audit_write` system stored procedure. Table 3-7 shows the detailed parameters accepted.
- ▶ The values for all parameters are user-defined and recorded in the audit log, when the event is fired.
- ▶ The `sp_audit_write` procedure can be called from a code module such as a stored procedure or trigger.
- ▶ In this example, it is called using a DML trigger as Listing 3-7 demonstrates.
- ▶ DML triggers should be used with caution as they can cause negative performance on a table with many writes.

Parameter	Description
@user_defined_event_id	Specifies the ID of the user defined event
@succeeded	Specifies if the event was successful. • 0 indicates that the event failed • 1 indicates that the event succeeded
@user_defined_information	Specify the description of the event

Table 3-7 `sp_audit_write` Parameters

Creating Custom Audit Events

```
CREATE TRIGGER FireCustomEvent
ON Sales.Orders
AFTER INSERT
AS
BEGIN
    IF (SELECT COUNT(*) FROM Inserted) > 5
    BEGIN
        EXEC sys.sp_audit_write 1, 1, 'More than 5 items order' ;
    END
END ;
```

Listing 3-7 Create a Trigger to Fire the Event



Summary

- ▶ SQL Server Audit provides a flexible and lightweight auditing mechanism to avoid issues of non-repudiation on privileged users.
- ▶ An Audit object is used to configure a target and specify behaviors of the audit. There can be multiple Audits on an instance.
- ▶ Server and Database Audit Specifications are used to define audited events. These are associated with a single Audit object.
- ▶ SQL Server Audit is extensible by the Audit Action Group `USER_DEFINED_AUDIT_GROUP` which enables custom events to be fired. The triggers are using the `sp_audit_write` system stored procedure which can be called from code module.